Bachelor Degree Project

The Use Of Software Product Lines in Game Development

Author: Abdulrahman Racheed
Author: Iker Garcia Martinez
Supervisor: Nadeem Abbas
Semester: VT 2025
Subject: Computer Science

Abstract

This thesis aimed to investigate the current state of Software Product Line Engineering (SPLE) usage in the game development industry. SPLE is a reuse-oriented development method known to reduce development costs & time; however, its adoption in game development is unclear. The research method employed was a mixed-methods survey approach, consisting of an online questionnaire and a focus group, as this thesis aimed to collect personal insights and knowledge from gaming professionals regarding SPLE. The analysis of the collected data showed a clear lack of knowledge and awareness amongst the less experienced developers, though interest in its adoption is high. Experienced developers identified benefits such as cost effectiveness, support for customization and modularity; however, they also noted structural rigidness, management complexity, and a high upfront cost as constraints that limit development flexibility. Both experienced game professionals and non-experienced game developers suggested the development of flexible and dynamic SPLE tools. While software product line knowledge and adoption are low, game professionals seem to agree that providing education or tools could help increase the use and adoption of SPLs in game development.

Keywords: Software product lines, Software product line engineering, Developers, Reuse, Games

Contents

1 Introduction

This thesis was made to provide knowledge regarding the use of the Software Product Line Engineering (SPLE) approach in game development. The video game industry consists of video games and their complex creation. Games are continuously growing in terms of graphics, complexity, consumers, and performance quality. Software development methods need to evolve in order to keep up with the demand. Among these methods, one stands out: Software Product Line Engineering.

Software Product Line Engineering (SPLE) offers a suitable approach by allowing the reuse of core assets across products. SPLE consists of two development phases: the domain phase, which focuses on creating reusable and flexible artefacts for tasks, and the application phase, in which those artefacts are used to create a family of products. Software Product Lines (SPLs) are software systems in a family of products that share and were built based on a set of core assets and features.

Research has proven that SPLE brings certain benefits, such as reducing development times of products, reducing cost, and improving product quality in terms of performance. Some of these benefits come from SPLE research [1]. Although SPLs provide a set of benefits, their application in game development was unclear. Game developers often produce games within a franchise or that have shared engines and assets. This makes the SPLE approach suitable due to the heavy emphasis placed on core software and asset reuse to streamline development and maintain a standard of quality.

This thesis explores the current state of Software Product Line Engineering in game development, how widely it is known, adopted, and the benefits & limitations thereof. To do that, we will employ a survey method for developers, which contains an online questionnaire and a focus group to collect the data needed to provide verified and reliable real-world data. This research aims to provide an understanding of SPLE's relevance within the game development industry.

1.1 Background

The basis of this thesis was centered around Software Product Lines (SPL) in the application area of game development. Furthermore, it also contributed to the broader research area of software architectures by addressing how SPLs could be used to improve code reusability and modularity across multiple software products. Video games have continuously evolved to the extent of being a part of an individual's day-to-day routine. Furthermore, Game Software Engineering (GSE) was recognized as an independent branch of Software Engineering (SE), which focuses on the development of video games [4]. Many software development methods have been implemented in order to produce and speed up the development of games, such as Agile Development, Clone and Own, and DevOps. Moreover, another concept that's used to help develop games quickly and efficiently is software reuse, which is a tactic that focuses on minimizing development time, investments, and resources by reusing existing software to create entire software systems [5]. Some approaches to software reuse contain Software Product Lines. SPLs are a software development approach that enables developers to reuse core assets such as graphics or game engines and functionalities across similar software products and families of products [7]. Developing games takes significant resources such as time and money and a strong focus on quality (e.g., performance, maintainability). Therefore, using a fast and cost-effective software development method that also offers performance quality benefits because of systematic reuse would be beneficial. Despite SPLs aligning with these needs, their adoption remains vague in the industry. Hence, this thesis aimed to investigate their usage and relevance in the game industry.

1.2 Related work

First paper

Problem: In [1], the authors addressed the problem of limited and error-prone code reuse in game development. Furthermore, for this they compared SPLs and Clone and Own (CaO) in developing game elements for *Kromaia* using Model-Driven Development (MDD) and Code-Driven Development (CDD).

Method: In this study the authors used a controlled experiment in which participants developed game elements using SPLe and others used CaO in MDD and CDD.

Results: The authors found that SPLs produced over 23% more correct results across both paradigms. In CDD, SPLs also outperformed CaO in efficiency (by 51%) and user satisfaction (by 13%). The contributions of this paper were the technical benefits of using SPLE for developing games.

Relevance: This study supports this thesis due to the performance improvements found in game development by using SPLE. Hence, it provided a technical foundation for this thesis to build upon.

Second paper

Problem: In [2], the authors addressed the difficulty of managing platform variability in game development, where they mentioned differences between hardware, performance, and input/output that make it challenging to maintain and build multiplatform games. The authors applied a component-based SPL approach using the PLUS method to develop multiplatform games.

Method: Their custom feature model handled variability in I/O devices, UI, and processing. They implemented this in a combat flight simulator using Microsoft's XNA framework, creating both desktop and mobile versions.

Results: Code reuse was 80.5% for desktop and 89.1% for mobile, showing SPLs can reduce development time and boost revenue across platforms.

Relevance: These findings aligned with this thesis as they claimed multiple benefits of SPLE, namely modularity and reuse.

Third paper

Problem: In [3], the authors address the issue of the increased complexity and demands in digital game development. The authors present a Model-Driven Game Development (MDGD) approach to raise the abstraction level in game development.

Method: They introduced three visual editors, namely Camera, Character, and Scenario. They allowed developers to model components and automatically generate code.

Results: An experiment comparing MDGD to traditional Java development showed that the MDGD group completed tasks in just 40.9% of the time. However, they were less familiar with the code, showing a trade-off between efficiency and code understanding.

Relevance: These results aligned with this research as they discuss core aspects of SPLE, namely abstraction and reuse, and how they influenced workflows as well as developer perceptions.

1.3 Problem formulation

The gap was identified as the unclear knowledge of SPL's use in game development. In the related work [1], the authors stated that the Software Product Line Engineering method may be overlooked when developing games, favouring other software development methods, without a clear explanation as to why that is the case. Bridging this gap is important, since SPLs have been proven to provide several economic and developmental benefits [1, 2, 3], as highlighted by other related works; however, the use thereof remained vague and unexplored.

To address the above-specified knowledge gap, four research questions (RQs) were formulated:

RQ1): What do game developers think about using the Software Product Lines Engineering (SPLE) approach for game engineering?

RQ2): What are the key challenges or obstacles in the use of the SPLE approach in game engineering?

RQ3): How can these challenges be addressed or mitigated?

RQ4): If given proper training for the use of SPLE, will game developers learn and use the SPLE approach for game development?

To address and answer the research questions, two main objectives were identified, which are the following:

1. **Understanding developer perception towards SPLE**
   By gathering quantitative and qualitative insights, this paper aimed to reveal how developers view SPLE in regards to complexity, relevance, efficiency, and practicality of SPLE in game development.
2. **Collecting and deriving recommendations that could possibly increase the adoption rate of SPLE in game development.**
   Based on findings from the previously mentioned research methods, the thesis aimed to propose recommendations that could support the use of the SPLE method in game development.

1.4 Motivation

This thesis contributed to the research area of game development by addressing the role of Software Product Line Engineering (SPLE) and how it improves code reusability, modularity, and efficiency. In game development, the reuse of core assets such as graphics, sound effects, and engines brings benefits such as a reduction in development costs-effectiveness, and a simplified multi-platform deployment [1, 2, 3]. However, the recognition and adoption of SPLE within the domain of game development remains vague and unclear, even with the benefits it provides for reusability and development. Thus, by investigating developers' perceptions and experiences with SPLE, this thesis aimed to provide valuable data that will benefit not only the game development industry as a whole but also researchers in the field of software architectures and game engineering.

1.5 Results

This thesis aimed to generate empirical and numerical results regarding the perceptions of Software Product Line Engineering in game development. Empirical data supported **RQ1** (*What are the key challenges or obstacles in the use of the SPLE approach in game engineering*) and **RQ2** (*How can these challenges be addressed or mitigated*), by providing previously reported obstacles and solutions.

The empirical data were gathered via descriptive results from our questionnaires and data from the focus group with game developers. The gathered data is considered Empirical because it was systematically collected from developers with direct experience and their perceptions of SPLE. These results aimed to capture the current industry's development practices, knowledge regarding SPLs, and any perceived barriers. Empirical data provided valuable insight regarding developers and their current practices.

The numerical results combined the insights gained from the empirical data and the statistical results from the questionnaire. It helped address **all research questions** by identifying trends, evaluating the impact of knowledge and training, and proposing potential solutions for a larger adoption rate within the industry.

1.6 Scope/Limitation

The scope set for this thesis was to understand the industry perceptions of Software Product Line Engineering in a game development context. This thesis did not analyze the technical aspects of SPLE or how to integrate such a method. The focus was instead set on understanding how the SPLE approach is viewed, whether or not it's adopted widely, and personal thoughts regarding the approach by developers. An analytical comparison with other software development methods was not expanded upon, although said methodologies were mentioned when context was needed.

Furthermore, another limitation was the sample size of the questionnaire/focus group. As students, we aimed to contact as many potential sources as possible, but with limited resources and access to industry professionals, the sample size was smaller compared to other research experiments. This could have influenced the results and introduced a sampling bias, but the responses nevertheless provided valuable insights that informed the industry about the current state of Software Product Line Engineering.

1.7 Target group

This degree project's target group was game developers. Regardless of their status, whether it be an employee at a large professional company, a standalone indie developer, or anything that falls in between. Anyone who works on or has worked on games is included within the scope. By addressing the previously mentioned research gap in 1.3 Problem Formulation and deriving suitable data, this thesis introduced the perceptions of Software Product Lines in game development. It addressed benefits such as cost-effectiveness and modular workflows that ultimately enhanced game development [1] [2]. Thus, any developer within the industry could benefit from this investigation.

1.8 Outline

The remainder of this degree project is structured in the following way.

- **Chapter 2-Method**, this chapter provides details regarding the research approach that was used. Furthermore, it also provides some considerations for reliability, validity, and ethics.
- **Chapter 3-Theoretical background**, the theme of this chapter was introducing the concepts of Software Product Line engineering and its relevance in game development.
- **Chapter 4-Results**, the objective of this chapter is to provide the findings of the survey method, which encompasses the questionnaire and focus group results descriptively and statistically.
- **Chapter 5-Analysis**, this chapter is responsible for presenting an analysis of the collected data using descriptive statistics and a thematic analysis to identify patterns or themes.
- **Chapter 6-Discussion**, the main objective of this chapter is to interpret the data gathered in relation to the research questions, as well as discuss study limitations and validity concerns.
- **Chapter 7-Conclusions and future work**, lastly, this chapter summarizes key findings, reflects on the study's limitations, and proposes a couple of directions that could be taken for future research.

2 Method

We followed a survey research method that contained both a Questionnaire and a Focus Group data collection approaches to achieve a comprehensive understanding of how Software Product Line Engineering (SPLE) is perceived and its use within game development.

The online questionnaire allowed for the collection of data from developers who had varied amounts of experience in game development. The said approach enabled us to identify common trends, challenges, and awareness regarding SPLE usage.

The Focus Group data collection approach, however, provided deeper insights into developer attitudes towards SPLE by allowing participants to share their experiences conversationally. It was conducted in person with a small group of willing participants.

Furthermore, relevant research papers were analyzed as part of a study review to support and validate findings from both the Questionnaire and Focus group. This review contributed to a theoretical foundation that identified key benefits of Software Product Line Use and disadvantages thereof. By conducting a study review, triangulating the collected data was simple and ensured that all conclusions made were based on developer opinions, whilst aligning with previous research.

2.1 Survey

To gather insights on the usage, awareness, and perceptions of Software Product Line Engineering (SPLE) in game development, a survey data collection method was utilized. This consisted of an online questionnaire, a semi-structured focus group, as well as a study review of related works. Combining said three approaches yielded both quantitative and qualitative results, which allow for a deeper understanding of the findings. This mixed approach aligns with the research objectives, addressing the existing pattern, why it exist, and potential mitigations. The subsections ranging between (2.1.1) - (2.1.3) will outline the strategy of the previously mentioned approaches.

2.1.1 Questionnaire

There was an online questionnaire created with Google Forms that was sent out to a plethora of game developers, which consisted of 32 questions. The communication channels used to recruit respondents consisted of:

- Social Media platforms (LinkedIn, Facebook, X (formerly known as Twitter)).
- Direct contact with studios through Email (Indie & Large studios).
- Academic circles (Peers, Professors, and University game programs).
- Developer communities (Subreddits such as r/gamedev, Discord channels).

The questionnaire was designed to collect quantitative and qualitative data about the use of Software Product Lines in game development from developers themselves, regardless of professional and/or experience levels. That was done to limit the bias from any one group of developers. It aimed to answer six main topics, namely:

- **Demographics**: Collection of demographic information from the participants, aiming to provide context to their responses.
- **Familiarity with SPL.** Assessing whether or not participants were familiar with the Software Product Line Engineering method.
- **Usage of SPLE in game development**. Determined whether or not participants have utilized the Software Product Line Engineering method to produce games.
- **Constraints of SPLE.** Participants were asked to explain perceived challenges or limitations of using the SPLE method.
- **Future of SPLE in game development.** Participants were asked to share their personal views on the potential for SPLE in the game development industry.
- **Considerations and opportunities for SPLE in game development**. This concluding section asked participants to provide reflective answers to how SPLE adoption could be improved, as well as the perceived benefits of using SPLE in specific contexts.

The structure of the questionnaire is shown in Figure 2.1

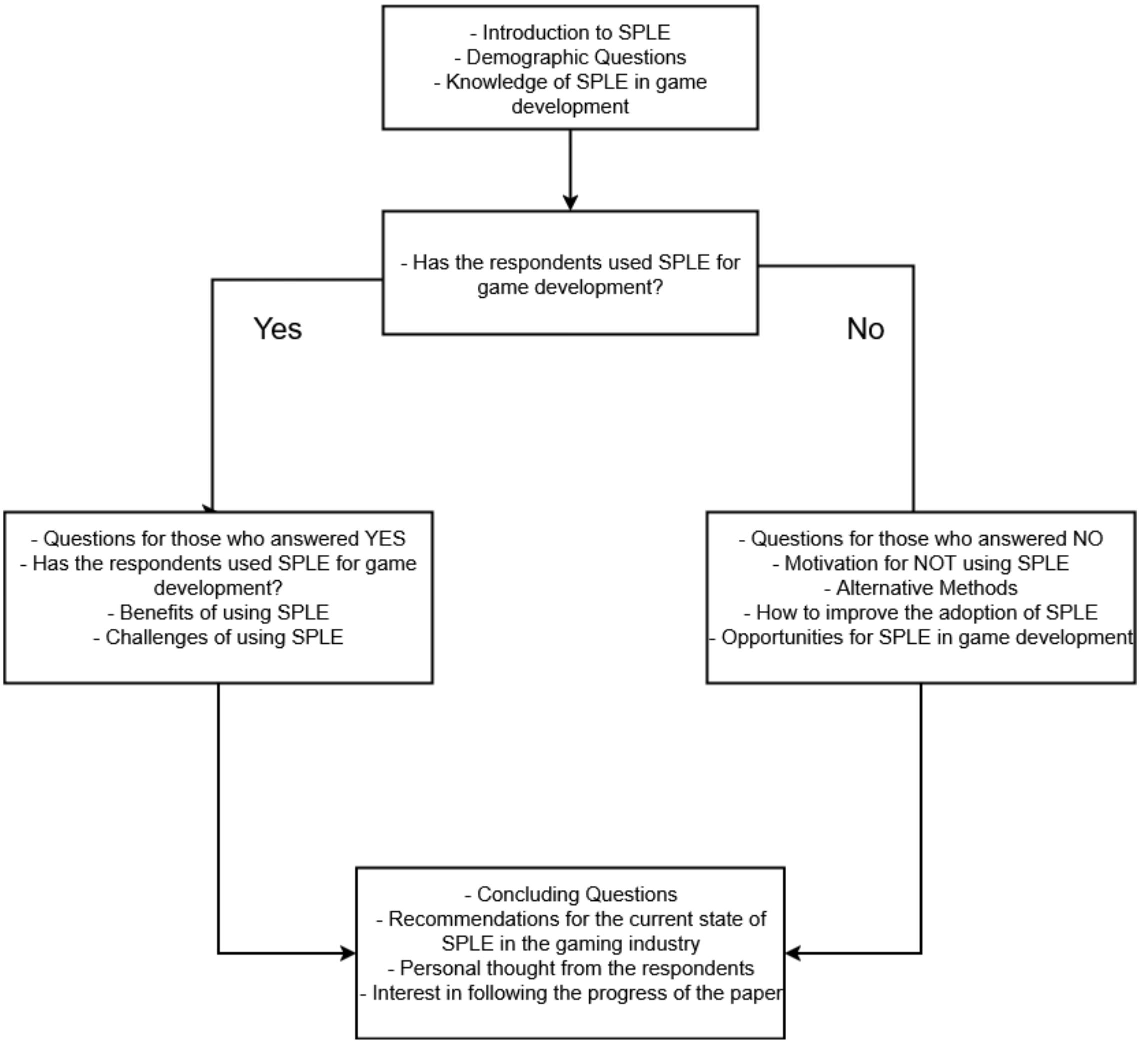

Figure 2.1: Visual representation of the Questionnaire flow

Introduction
The questionnaire begins by introducing the respondent to the concept of Software Product Line Engineering (SPLE), explaining the two-phase structure that exists in it (Domain & Application Phase). This foundation ensures that all participants respond to the questions with a shared understanding of SPLE.

Demographics Section
This section collected basic demographic information about the respondents. Questions included:

- Job Title / Job Role
- How many years of experience
- Team size
- Genre specialization
- Used Game engines

SPLE usage filter
Respondents were then asked about their familiarity with SPLE engineering and whether or not they had used that approach in game development in the past. Based on the response, the questionnaire then branched into two sections.

For respondents who have used SPLE (Left Branch)
The respondents were asked to answer and provide motivation on questions regarding their experiences with Software Product Line Engineering within game development. The covered topics were:

- Motivation for utilizing SPLE
- Their perceived benefits
- Their perceived challenges and drawbacks
- Constraints encountered when integrating SPLE into the codebase
- A combination of SPLE with other development methods

For respondents who have not used SPLE (Right Branch)
If the respondents have not used SPLE in game development, they were redirected to this section. They were prompted to share their thoughts on the following topics:

- Motivation for **not** utilizing SPLE
- Their current development methods, detailing whether or not they use specific methodologies, tools, or practices for development.
- Which limitations their method introduce
- If they see any opportunities or benefits of SPLE
- If they have any interest in using SPLE in the future

Concluding Considerations and Thoughts
Lastly, respondents were prompted to answer concluding questions for the questionnaire.

- Interest in receiving the final report
- Willingness to participate in a follow-up interview
- How confident are they in the responses given
- Any concluding thoughts or input regarding SPLE

Note: The questionnaire questions are provided in Appendix A for reference.

2.1.2 Focus group

A focus group is a research method that collects a small number of participants to talk about a certain topic in an open discussion. While questionnaire results offer structured data, focus groups allow for open-ended discussions where participants can discuss their reasoning, understanding, and challenges with each other's ideas. This method is useful for understanding perceptions and attitudes toward a subject that could be missed in more structured data.

This degree project organized a focus group with six game developers with varying levels of experience from Linnaeus University. This process was a semi-structured interview about the implementation, considerations, and thoughts of Software Product Line Engineering in game development. It was conducted in approximately 2 hours.

The focus group had the purpose of bringing knowledge about the SPLE approach to those developers who have not utilized it before, while also giving those who have a space to further discuss and share their experiences. This approach not only allow developers to share their personal insights, but also served as an educational opportunity to learn about SPLE implementation in game development. Furthermore, all data gained from the focus group was used to validate and complement findings from the questionnaire by providing context to the answers.

Figure 2, as shown below, illustrates the flow of the focus group where Q represents the questions asked to the participants.

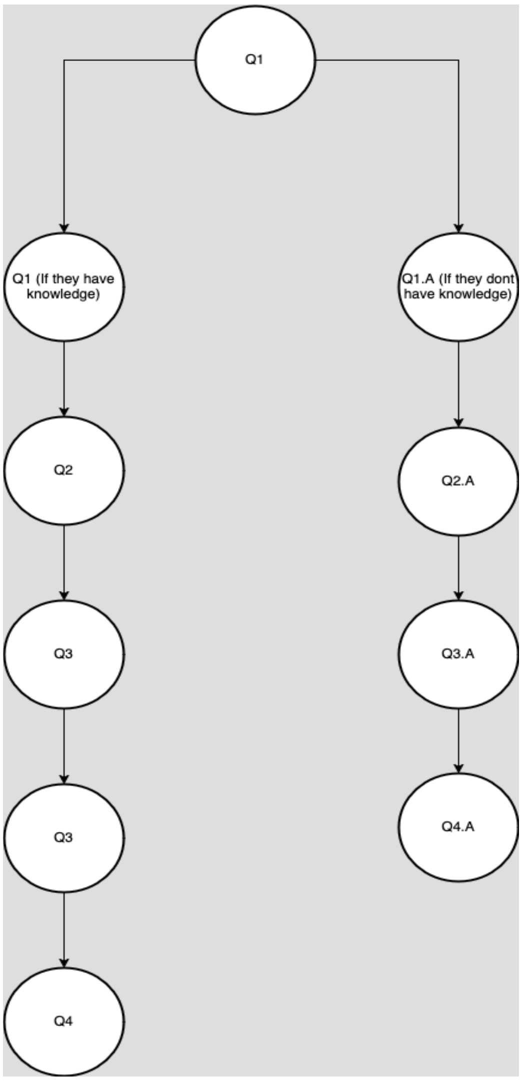

Figure 2.2: Visual representation of the focus group flow

The focus group started by asking developers if they have any knowledge regarding SPLs; depending on their answers, the flow will change. If the respondent knew what Software Product Lines were and how they could be used in game development, the developers were asked about the following:

- If the developers have implemented SPLs on any game they were a part of developing.
- Which challenges are associated with the SPl adoption?
- Whether or not the developers have used differing development methods.
- Whether or not they would personally recommend Software Product Line Engineering.
- And whether or not they see a possible increase in SPL usage for game development in the future.

In case developers answered that they did not know, then the following questions were asked.

- What the developers think are the main challenges of SPL adoption in game development, based on the knowledge provided.
- If allowed to learn more about SPLs, would they seize it?

While the game developers were answering and discussing with their peers, we noted anything that stood out. This approach captured personal insights that could otherwise be missed, including unique challenges, existing knowledge and perceived barriers to implementing SPLs in game development.

2.1.3 Study review approach

This approach reviewed existing literature, addressing the application of Software Product Line Engineering in the context of the gaming industry. Relevant data such as benefits, limitations, and alternative methods to the SPL approach, was collected. This ensured that the structure and contents of the online questionnaire and interview questions stay relevant.

2.2 Analysis methods

To ensure the validity of results from both the questionnaire and focus group, two analysis methods were selected, namely descriptive statistics and thematic analysis.

2.2.1 Descriptive Statistics

Quantitative data from the questionnaire were analyzed with the descriptive statistics analysis method. This strategy visualised the responses in terms of percentages, averages, and statistical representations. Thus, identifying common patterns, such as which benefits are most recognized, was simple.

2.2.2 Thematic Analysis

The qualitative data from both the open-ended questionnaire responses and focus group discussions were analyzed using the thematic analysis method. It involved reviewing the data, identifying common and recurring ideas, and grouping them into themes (e.g., “Lack of awareness”, "Perceived Benefits”). This method was chosen because it is well-suited for identifying patterns in varied types of responses.

2.3 Reliability and Validity

2.3.1 Reliability

To ensure reliability across the data gathered throughout the different survey methods i.e. questionnaire and focus group, the participants were selected within the game development community. Furthermore, the data was collected from multiple participants, either working in organizations or not, so as to reduce the risk of bias or isolated perspectives.

2.3.2 Validity

To ensure validity, a multi-method strategy was employed. Firstly, the data was cross-examined through triangulation, comparing insights from the Questionnaires and the Focus Group to ensure consistency between all research methods. Lastly, through conducting a literature review, we aimed to support the findings from the research methods, helping to draw consistent conclusions that were backed by research.

2.3 Ethical Considerations

Based on the research methods previously discussed in section 2 Method, several ethical considerations needed to be addressed for this thesis to have some ethical grounds. The considerations for the methods are as follows:

- **Data Collection/Storage**: The collection of data varied between methods. For the questionnaires, the data that was collected from the participants was going to be stored in Google Forms, which is only accessible by authorized parties. In this case the authors and supervisors of this thesis. The data collected from the focus group was safely kept in personal devices to prevent leakage and protect confidentiality amongst the participants.
- **Confidentiality**: When it came to privacy for the questionnaire, Participants were notified at the beginning of the questionnaire and in the invitation email that their answers would be kept private and anonymized. In the focus groups all respondents were told at the beginning that their answers would be kept safe.
- **Anonymity**: For anonymity in the questionnaire the participants were notified both in the invitation mail and at the beginning of the questionnaire that their answers would be kept confidential and anonymous. Lastly, it is similar in the case of the focus groups where all the participants gathered were given a chance to look over their answers in case they wanted to change them or if they simply did not wish to have it exposed, they were told what their results were for and that they would be kept anonymous.
- **Consent**: Participants were asked for their consent in the invitation emails for the questionnaire. For the focus group, however, all participants were asked if they were willing to participate in the session, and their participation only continued after consent was given.
- **Bias**: There was a risk of biased answers since it was assumed that some of the participants either used the software product line approach or did not, so there was going to be a mix of data favoring one method over the other.

3 Theoretical Background

3.1 Software Reuse

Software Reuse is described as: "*The process of creating software systems from existing software, rather than building software systems from scratch*" by Charles W. Krueger [5]. Thus, by leveraging reuse in software development, products can be built with better quality, faster time-to-market, improvements in performance, and better productivity in developers [6]. Reuse is an essential strategy in software engineering due to the fact that building each component from scratch is very time-consuming and inefficient when alternative methods exist. There are multitudes of Software reuse techniques, such as Application generators, Component/Artefact libraries, and domain-specific languages.

3.1.1 Systematic Software Reuse

Systematic software reuse is a formal implementation of software reuse that emphasizes planned and structured methods for integrating reusable components between projects. It introduces clear documentation, a structured process and tools that ensure that reusability is maintainable and consistent throughout all projects.

3.1.2 Software Product Line Engineering

Software Product Line Engineering (SPLE) is a well-documented approach to systematic software reuse. It is defined as: "*a paradigm to develop software applications (software-intensive systems and software products) using platforms and mass customisation*" by Phol, Böckle, and Linden [7]. The stated paradigm contained a dual-process structure, which was composed of two phases, namely Domain and Application engineering.

Domain Engineering Phase	Application Engineering Phase
The domain engineering phase provided process guidelines to develop a system or application-independent platform. The platform consisted of all types of reusable software artefacts, including: ● Software Requirements ● System and Architectural Design ● Artefact Implementations (engines, assets, libraries, etc.) ● Tests cases	The Application Engineering phase provided process guidelines for building software products by selecting, reusing, and implementing core assets identified in the domain engineering phase. This ensured that each product developed meets a certain set of criteria by ensuring that all identified artefacts equally meet the requirements.

Table 3.1: A description of the dual-approach in SPLE.

3.2 Game Engineering

Game engineering refers to the development of video games using software engineering techniques adapted to this specific purpose. Unlike traditional software, video games combine technical systems with creative elements, such as writing, music, visual graphics, and player interactions. Thus, the development process must support functionality and creativity. The nature of game engineering requires fast development, maintainable implementations, and flexible designs to match the creative aspects of game engineering [6]. This field encompasses a wide range of developers, from large-scale game studios (also known as AAA studios), to small independent (Indie) teams.

3.2.1 Game Engineering Approaches

Game engineering uses a variety of software development methodologies, adapted to suit the need of creativity in video games.

- **Clone and Own (CaO)**: This is a reuse-based method where developers copy and modify existing assets from previous products to create something new. This method, whilst simple, can introduce overhead and inconsistencies across products over time [1] [8].
- **Model-Driven Development (MDD)**: In MDD, developers can use visual tools to help define artefacts (e.g., characters, enemies, items) and automatically generate the code. This method is aimed at improving productivity and reducing implementation errors. However, this can lessen familiarity for developers, which may complicate maintenance and debugging [1].
- **Agile development methods**: As noted in [9], agile development methods are widely accepted within the game development world. The iterative-based structure of agile development is effective for managing the creative needs of game development, allowing for easy flexibility and adaptability.

3.2.2 Challenges in Game Engineering

- **Technical Debt**: In video game development, technical debt can rise due to short-term solutions that will make future changes very difficult, or even impossible to implement. The work of Klara Borowa and Andrzej Zalewski revealed numerous reasons that contribute to technical debt [10], some of which are:
  - **Planning**: Due to the creative and changing nature of game engineering, planning can become very difficult, resulting in time pressure on developers, which can incur decisions that affect the technical debt.
  - **Lack of Documentation**: Developers rarely document properly due to the increasing stress to develop new features, which creates dependencies on developers to explain the code's functionality. This established technical debt as well.
  - **No clear methodology for game development**: There is no clear way to develop games, and the existing literature does not provide a clear understanding of the challenges in game engineering. While methodologies such as Clone and Own (CaO), Model-driven development (MDD), and Agile are used, they are adapted to fit game development’s creative nature. This, in turn, also creates technical debt.

3.2.3 Software Product Lines in Game Engineering

Software Product Line Engineering (SPLE) offers a structured approach to game engineering by introducing benefits to efficiency, modularity, cost effectiveness and multiplatform deployment [1] [2] [3]. A SPLE process in game development could then look like this:

- **Domain Engineering**: Creating reusable core systems, which may include:

    - Reusable game mechanics (characters' movements, collision)
    - Reusable game engines
    - Asset library (graphics, sound effects, animations)
- **Application Phase**: By using the reusable core systems, developers can then create a product line of games with similar core logic. For example:
    - A 2D platformer game and a 2D metroidvania share similar physics and movement systems.
    - A set of games from the same franchise with varying storylines, but built on top of the same game engine.

Therefore, it enables game engineers to consistently create games while potentially reducing the overall technical debt by introducing a structured and efficient methodology such as Software Product Line Engineering.

4 Results

This chapter presents the findings from the data collected through both survey methods, the Online Questionnaire, which contained 16 participants, and the Focus Group, which composed 6 participants. The results were derived from participants' responses to the questions presented in the questionnaire and questions from the focus group, respectively. Furthermore, they were stored in a Google Drive folder only accessible to the authors of this thesis. All the findings are divided into the following three sections:

- Numerical Presentation, which presented the statistical data using table graphs depicting questions from the questionnaire with multiple-choice answers. The graphs are structured in the following way: they list what question of the questionnaire it is, they present the answers the respondents could choose from, and how many people selected said answer (e.g, Q1.1 project manager(1)).
- Descriptive Answers provided an understanding of detailed answers to open-ended questions within the questionnaire.
- Focus Group Findings, which provided insights based on an open group discussion with participants with differing backgrounds.

All questions present in the questionnaire and in the graphs are attached in the Appendix. Please refer to it when reading the results.

4.1 Questionnaire Results - Numerical Presentation

4.1.2 Demographic

Figure 4.1: Table graph for demographic questions

Figure 4.2: Table graph for demographic questions

The questionnaire begins with some demographic questions to gauge how experienced the game developers were before starting to gather data regarding Software Product Lines (SPLs) in game development. The previous Figures 4.1 and 4.2 depict all questions from the questionnaire that were used to determine developers' experience and what answers the respondents chose. For all the graphs, the number in parentheses refers to the number of people who selected the corresponding answer (e.g, unity(11))

4.1.2 SPL usage filter

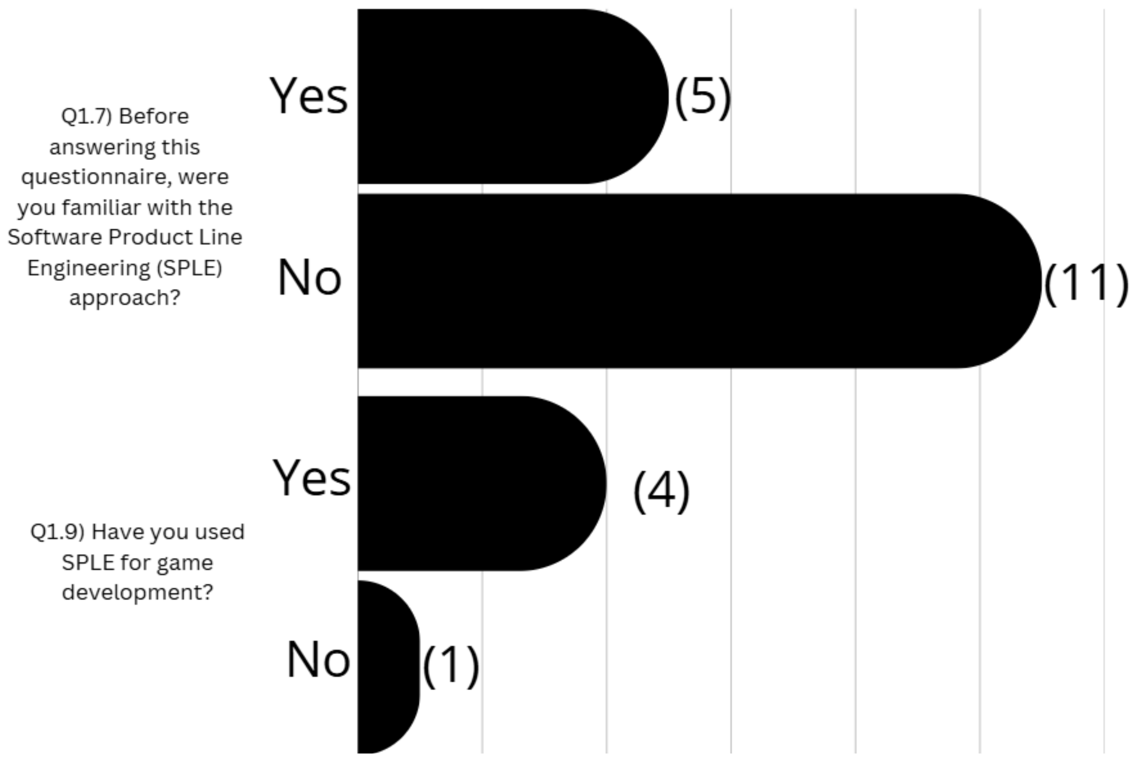

Figure 4.3: Table graph for SPL usage filtering questions

The two questions from Figure 4.3 focused more on finding the average number of developers who were familiar with the SPLE approach and if they used it before to make games. Depending on the answer, the flow of questions splits into questions for those developers who have previously used SPLE for developing games and questions for those who have not used it or were not familiar. In Q1.7 from the people who answered yes, 60% of them were experienced developers. In Question 1.9, 80% indicated that they have used SPLE for game development, whilst 20% indicated that they did not. Question 1.8 is present in section 4.2.

4.1.3 Developers that have used SPLE for game development

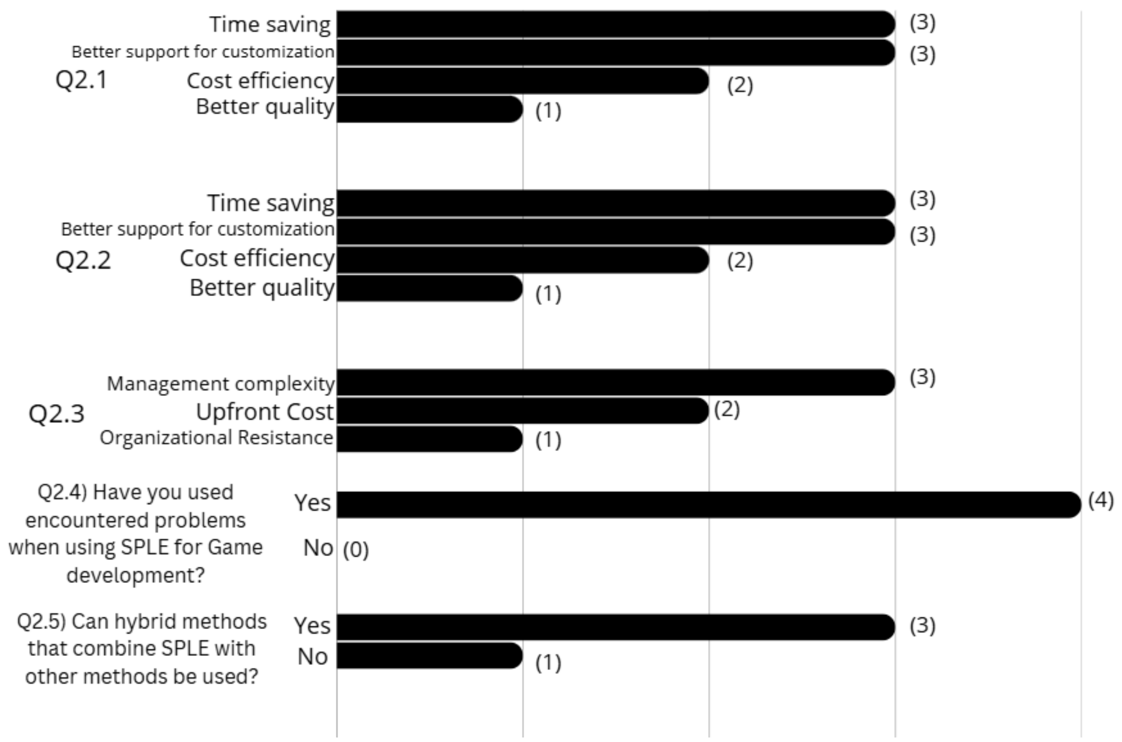

Figure 4.4: Table graph for SPL use in development questions

The questions represented in the Figure 4.4 were targeted for those developers who have previously used SPLE for developing games, some benefits they could identify, possible challenges, and overall thoughts. Questions 2.4b and 2.5b are not shown in the graphs since they were more open-ended to answer, and the results for said questions are found in section 4.2

4.1.4 Developers that have not used SPLE for game development

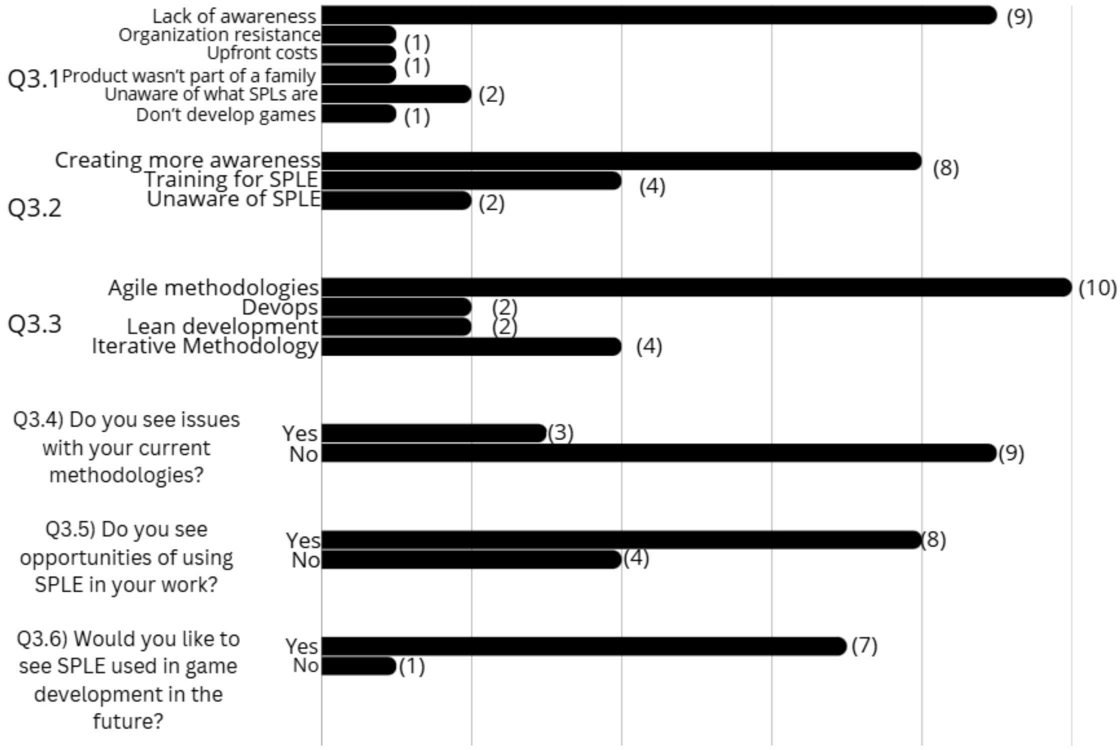

Figure 4.5: Table graph for developers who do not use SPLs

The questions represented in Figure 4.5 were the ones targeted for developers who have not previously used SPLE for developing games, but have used another development method. For question 3.1, some of the results for others were: "project was a single arcade game, not a product family or series of similar games", "We develop PoCs and Demos. Could be good for production". In question 3.2, the results for others were: "I don't know what it is" and "I don't know". Questions 3.4b, 3.5b, 3.5c, 3.6b and 3.6c are located in section 4.2 since they were more open ended questions.

4.1.5 Concluding thoughts

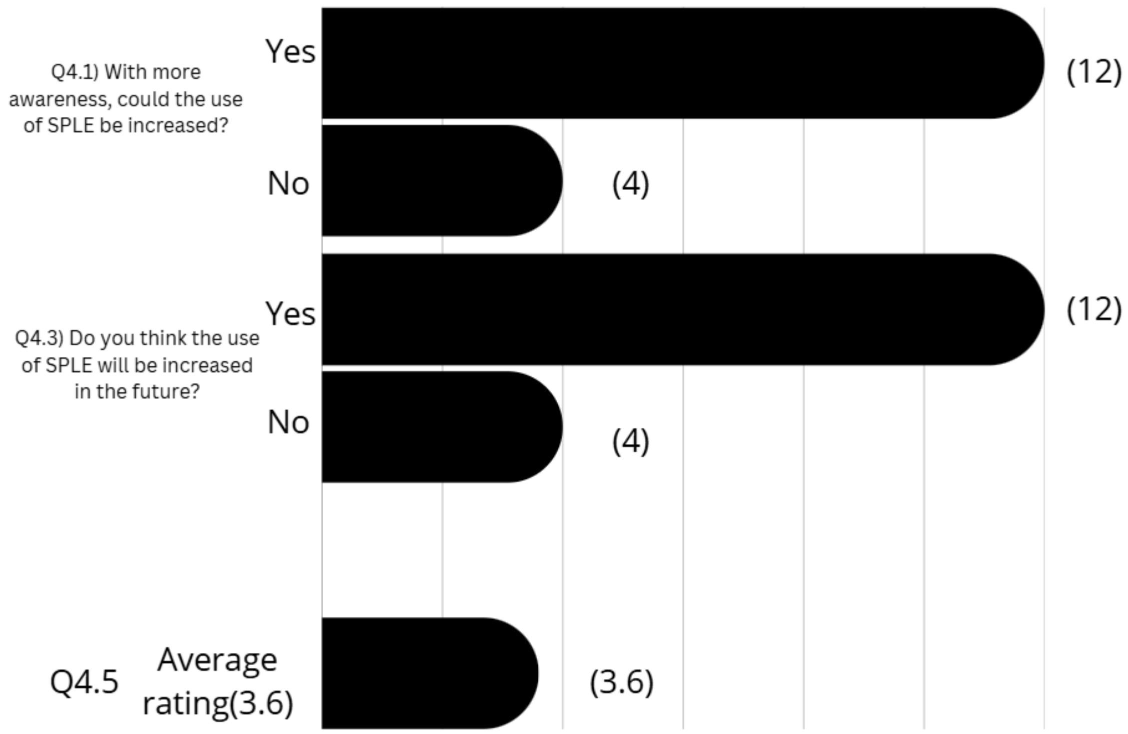

Figure 4.6: Table graph for Concluding thoughts

Figure 4.6 depicted that 66% of developers believed that SPLE adoption could increase in the future if there was increased awareness. Furthermore, this questionnaire received an average confidence rating of 3.6 out of 5. This showed that whilst there may be some uncertainty in answers given by respondents, it still reflected a generally medium to high level of confidence in their answers.

4.2 Questionnaire Results - Descriptive Answers

Q1.8) Please briefly describe your understanding of SPLE.

In total, five participants had an understanding of SPLE. There was a common pattern in their answers; 100% of them mentioned the reusability aspect of the SPLE approach by identifying components and reusing core assets, methods, or software in a family of products, which means that they had a basic understanding of the principles of SPLE regardless of experience.

Q2.4b) What problems or issues did you, your team or company encounter when using SPLE approach for game development?

A total of four respondents answered this question. 50% noted that integrating Software Product Line Engineering introduced a high initial investment due to the time and costs required to set up a flexible and automated product line. Furthermore, the same 50% of respondents also noted that managing the complexity in variability is challenging due to the complex visual and interactive game elements and their variability between products. Moreover, they believe that not enough support exists for SPLE tool integration, requiring a customised workaround.

Two respondents noted that reusable components from a product line might not always be optimized for specific games, which might introduce performance issues. Ensuring consistent use of product line assets in a larger workspace can be difficult, said one respondent.

Q2.5b) Please describe in what way you think that the use of core SPLE principles can be combined with other game development or engineering methods.

Only two respondents answered this question. The first respondent (50%) noted that the way they think about how the core principles of SPLE can be combined with other methods is with assets and commonly used functionalities. The second respondent (50%) however went into a deeper response, showcasing how they could be combined with agile development, model driven development (MDD) and component based software engineering by providing real-world examples.

Q3.4b) What limitations, problems, or issues have you or your team have experienced with your current engineering or development methodologies for game development?

Respondents shared that team coordination was a major issue. Certain tasks and solutions might overlap, thus creating conflicts when proceeding to the next step of development. Furthermore, absence of employees leads to problems of un-finished development plans.

Q3.5b) What opportunities or benefits do you see or think of for use of SPLE for game development in your work?

Some of the respondents shared similar benefits highlighting that SPLE would be beneficial in case they needed to reuse assets, code and components. Whereas the rest noted that it would help with work, such as learning new skills, improving workflow, and increasing efficiency.

Q3.5c) Why do not you see or think of any opportunities or benefits of using SPLE for game development in your work?

For this question, the respondents had a mixed response. 50% of the respondents identified that the rapid change of game design and high creativity demands make the structure of SPL seem less adaptable or beneficial. The other 50% simply noted that they are not familiar enough with SPLE to answer this question.

Q3.6b) Why would you like to use the SPLE approach for game development in future?
The majority of respondents noted that the benefits SPLE brings to Modularity, Efficiency and Code Quality is their motivation. One respondent indicated that SPLE could offer support for a series of games with multi-platform deployment efficiently.

Q3.6c) Why would you not like to use the SPLE approach for game development in future?
Only one (100%) respondent answered this question. They show hesitation to the integration of SPLE in their future development.

Q4.1b) Why do you think that the use of SPLE in game development can not be increased even with more awareness and exposure?
Two (50%) respondents noted that due to the game industry's focus on innovation, creativity and frequent design changes, rigid and standardized core assets from Software Product Lines might limit the overall adoptability of SPLE, regardless of increased awareness.

Q4.2) What other suggestions or recommendations do you have for improving the use of SPLE in the game industry?
Of all respondents, ~64% of the respondents chose to leave out any suggestions. The latter ~36% decided to provide some personal opinions such as spreading awareness of SPLs, making it more accessible for beginners, and developing dynamic tools that allow for flexible SPLE integration. The respondents noted these reasons as possible suggestions to improve SPLE usage in game development.

Q4.4) For the above questions, explain why you think that the use of SPLE in game engineering is going to be increased or not to be increased.
Respondents showed mixed views on the future of SPLE in game development. Some think that adoption might increase due to its benefits for multiplatform development, clarity, scalability, and effectiveness, especially in mobile gaming. Others were less optimistic about the future of SPLE, noting that the rigid structure conflicts with the industry's emphasis on speed, creativity, and customization. Some participants see the potential of SPLE, but recognized that they have a lack of understanding to confidently provide an answer.

4.3 Focus Group results

The results presented in this section were gathered from an in-person focus group of developers, where they shared and discussed their personal knowledge when questioned about SPLE. There were a total of 6 developers present in the focus group. The data derived from the focus groups was later analyzed against the questionnaire data. The developers questioned in this focus group differed from the ones who answered the questionnaire.

Questions	Findings
Q1. (**If they have knowledge of SPLE**) Have you ever developed or helped develop a game with SPL's and if so, what benefits would you describe were brought by it?	Only two participants (33%) noted that they had in fact used SPLE before and some of the benefits they noticed were that SPLE helps reduce complexity when developing a game with it and reduces time of development while also saving resources such as budget. The latter 66% noted that they have no prior experience of SPLE.
Q2. What would you say are the biggest challenges or obstacles when using SPLs, or any challenges you came across personally?	Here, the participants mentioned that there could be optimization issues, such as performance overhead, and difficulty of managing variability in a family of games. If there is an error in a reused component, then it is difficult to implement a solution that fits all games, since that component may be used across a family of games.
Q3. Have you ever used other software development methods and if so then what would you say are the differences and benefits between SPLs and this other method?	Four participants (66%) had used different methods and they identified some benefits in their methods: efficiency of developing, flexibility for customization without major redesign and provided ease of integration for game development
Q4. Based on your experiences and preferences. Would you recommend the use of SPLs on game development? Why or why not?	Two participants (33%) recommended the use of SPLs due to its reusable nature. One participant favored it due to the fact that it presents a clear solution to new developers with bare knowledge of game development. Another participant noted that they personally would not recommend SPLE due to their belief that it brings dead and useless code into the codebase, which is code that is never executed or offers no functional contribution to the software.

	Thus confusing the developers unnecessarily.
Q5. Do you see SPLs being implemented more in the future, why or why not? please illustrate based on your preferences.	~83% of the participants noted that they might see an increase of SPLE usage in the future. The reason for this is that a lot of gamers complain about unoptimized game releases. By utilizing SPLE, and creating already optimized and reusable assets such as physics or graphics assets, optimization issues can be avoided.
Q1.1 (**If they don't have knowledge of SPLE**). If there was an opportunity online to learn about SPLs and their usage in game development, would you seize that opportunity and learn more?	100% of respondents shared that they would be interested to learn more.
Q2.1 (They are willing) Would you say that if with enough exposure or education will developers be more inclined to adopt SPLs or learn how to integrate them into their work?	Depends on the type of education. They shared that competitive challenges or “game-jams” with a focus on using SPLE would bring a lot of awareness for SPLE and could potentially increase use thereof. SPLE could be integrated in a smaller-scale work, specializing in one specific aspect of the codebase such as physics, or graphics, rather than everything.
Q3.1 Based on preliminary information and assumptions, what do you assume are the main challenges are and how would you address them?	One participant interestingly noted, “There could be a lack of personality in games due to a lot of assets being re-used. The same logic or same assets in games could lead to low personalization”. Furthermore, the focus group agreed that SPLE can bring unnecessary code and introduces bloating into the codebase because SPLE favors inclusion of reusable assets over creating everything from scratch, which leads to unused features and extra configuration logic.
Q4.1 After you learn more about SPLS, would you adopt them for future work? If so, would you also recommend it to other developers?	100% of the group said yes. Whereas one participant noted that due to the benefits it brings, integrating SPLE into game development would be very beneficial overall.

	Another participant highlighted that the SPLE solution could differ in performance depending on the scale of the games being developed. That respondent noted that larger games would introduce a lot of variability in components, thus they would recommend this solution only to smaller-scale games.

Table 4.6: Questions (Left), Results (Right). Representation of descriptive results from the focus group.

5 Analysis

To analyze closed questions, the descriptive statistics analysis method was used. It presents data in 5.1 Descriptive Statistics using percentages, averages and statistical representation of results. However for open ended questions, a thematic analysis approach was used. Its purpose was to identify and analyse common patterns found in the answers, which can be found in 5.2 Thematic analysis.

5.1 Descriptive Statistics

5.1.1 Demographics

Q1.2 showed that 31% of respondents have worked with game development for 1-3 years. Moreover, another 31% noted 4-8 years of experience. Lastly, 12% noted 9-20+ years of experience. The remaining participants had either less than 1 year of experience or were hobbyists.

Q1.3 showed that 81% of respondents have developed between 1-5 games, whilst 19% reported 10+ games.

Q1.4 revealed that 69% of respondents worked in teams containing between 1-10 people, 19% working in teams containing between 11-50 people, and only 12% working in teams of 50+ people. Thus, **Q1.2**, **Q1.3,** and **Q1.4** all highlighted that respondents of the questionnaire were typically mid-level developers with a focus on less-experienced developers who do not have a lot of experience.

In **Q1.7,** when asked about prior knowledge regarding Software Product Line Engineering, only 31% answered yes, whilst the latter 69% reported no previous knowledge. There was even a knowledge gap between the more experienced developers that participated in this questionnaire. This could indicate that familiarity with the Software Product Line Engineering approach did not necessarily link with what tools are used, but rather an overall lack of knowledge.

5.1.2 Developers that have used SPLs for game development

Results of **Q1.9** showed that out of five developers who have prior knowledge of Software Product Line Engineering, 80% have practical experience of using it in game development projects.

Q2.2 revealed that 75% of respondents indicated that SPLE brings better support for customization and that it saves development time. Furthermore, 50% noted cost efficiency as a byproduct. Only one (25%) indicated quality improvement. No developer mentioned consistency as a benefit.

These results aligned with findings from previous research [1, 2, 3], which state that SPLE in game development contexts could improve time-to-market and in efficiency, strengthening the validity of claims made by respondents.

5.1.3 Developers that have not used SPLs for game development

Q3.1 asked why developers believe they/their team have not used SPLE for developing games. A total of 75% of respondents indicated that the main reason was due to a lack of awareness. Furthermore, the remaining 25% detailed reasons such as high initial investments, organizational reluctance, lack of knowledge and lack of necessity. The data could indicate that the primary obstacle of SPLE adoption in game development might not be the technical complexity, but a lack of knowledge of the methodology itself

In **Q3.2,** when asked what could increase the use of SPLE, 67% believe that creating more awareness about SPLE would help boost the use of it in game development. The remaining 33% believe that training could increase the use, or that the respondents are unsure of how.

Thus, results from **Q3.1** and **Q3.2** showed that lack of awareness and knowledge with SPLE was a recurring barrier, notably amongst less experienced developers. This suggests that there was a need for the exposure or education of SPLE methodologies within the game development industry.

Q3.3 asked developers to share the development methods they currently use. The majority of respondents (83%) reported using Agile methodologies. Iterative & Incremental Methodologies were also reported to be used to a lesser extent (33%), whilst DevOps(17%) and Lean Development (8%) were less commonly used. The major preference for Agile methodologies shows the nature of game engineering, which favours flexible, adaptable and fast development methods for game development. This could suggest that there is an obstacle for SPLE adoption in game development due to SPLE's structured nature and people's perception of it.

In **Q3.4** & **Q3.4b**, Developers were then asked to share whether or not they experienced any limitations, problems or issues with their current development methodologies. 25% reported issues such as team coordination and tool limitations. The remaining 75% did not perceive any notable problems.

Q3.5 & **Q3.5b** showed that 67% of respondents believe that there are future opportunities for adopting the SPLE methodology due to the benefits it brings to reusability, maintenance, and into the workflow. This indicated that developers who did not experience any significant limitation in their current workflow could identify the value of SPLE in game development.

5.1.4 Concluding thoughts

Q4.1, Q4.1b and Q4.2 highlighted that 75% of respondents believed that there could be an increase in SPL adoption in game development if enough awareness exists. The remaining 25% reported that regardless of increased exposure, the rigid structure of SPLE reuse could conflict with the dynamic and innovative nature of game development [4]. Approximately 31% of respondents also noted improvements that SPLE could introduce, including new dynamic tools, intuitive marketplaces, better accessibility, and awareness.

Furthermore, **Q4.3** and **Q4.4** showed that 75% of all respondents believed that SPLE usage would increase in the future. The perceived reasons that suggested an increase in SPLE usage were:

- Intuitive solution for software reuse.
- Efficiency in handling scalability and reusability.
- Multiplatform development potential.
- Brings clarity into the workflow.
- A growing and modular asset pool.

However, the remaining 25% that do not perceive an increase believed so for the following reasons:

- A structured and rigid methodology such as SPLE does not allow for the innovative, unique and fast paced solutions that game development needs.
- Lack of support from pre-built game engines.

5.2 Thematic Analysis

To analyze the gathered qualitative data, a thematic analysis was conducted. It involved reviewing open-ended questions and identifying commonly occurring themes, whilst grouping them into codes (categories). The following codes were identified based on relevance to the research questions and how often similar answers came up.

- Lack of Awareness & Knowledge.
- Perceived benefits of SPLE.
- Adoption Challenges.
- Potential for future use.
- Suggestions for improvements.
- Relevance to game development.

5.2.1 Lack of Awareness & Knowledge

Descriptive results in section 5.1.3 showed that the majority of respondents had never heard of the SPLE approach or did not understand it well enough to adopt it. This theme was notably common among less experienced developers. Some focus group participants strengthened this notion, reporting that SPLE was not covered in educational media.

However, even when participants were unaware of the SPLE methodology, they understood the core principles of the SPLE approach, especially its focus on reuse. One focus group participant stated:"*Isn't the unity asset store basically a product line?*". This highlighted that whilst the SPLE approach may not be known amongst less experienced developers, core principles were present in the way developers perceived and developed reusable game assets.

5.2.2 Perceived benefits of SPLE

Respondents who have used the SPLE approach for developing games highlighted several developmental benefits of its use. Those benefits included **cost efficiency**, **time saving**, **better quality** and **better support for customization**. All said benefits were realised by the respondents. Moreover, focus group participants highlighted that the SPLE methodology saves development time and simplifies it, stating: "*If you have an idea, it's easier to create an abstract implementation rather than developing it iteratively*".

5.2.3 Adoption Challenges

Participants who have used the SPLE approach were also asked to identify challenges of adopting Software Product Lines. Common challenges include **management complexity**, **high initial costs**, and **organizational resistance**. This proved that whilst SPLE is a great solution for component reuse, it came with a set of trade-offs. The structured solution may not align with development needs, low budgets may not cover the integration of such an approach and management teams may be reluctant to adopt it, opting for a simpler solution.

5.2.4 Potential for future use

Whilst the low awareness rate proved to be a barrier of adoption, especially for less experienced developers, many respondents noted interest in exploring Software Product Lines in future endeavors. In addition, the more experienced developers highlighted the potential SPLE holds in regards to multi-platform development, mobile gaming and game franchises. One participant stated "*I think SPLE approach is very suitable for mobile games. With the increase of mobile gaming SPLE usage can be increased as well"*.

These findings aligned with the paper *Applying software product lines to multiplatform video games* by Emad Albassam and H. Gomaa [2], which highlighted how variability of devices affects Software Product Line Engineering.

5.2.5 Suggestions for improvements

A common theme between both survey methods was that respondents suggested awareness campaigns, training opportunities, and tools that better support SPLE usage. Furthermore, one participant from the focus group suggested online game development competitions, such as the game-jam challenge, but with a focus on using SPLE. Moreover, both survey methods revealed that many participants believed that if these improvements were realised, developers would more often use SPLE for game development, especially newcomers.

These results indicated that game developers, regardless of knowledge or experience, had similar ideas on how to spread awareness of SPLE and that it could lead to an increase in use amongst all developers.

5.2.6 Relevance to game development

A subset of experienced developers questioned the effectiveness of SPLE in larger-scale game projects. Their insights suggested that game development projects prioritized speed, innovation, support for creativity, and that a structured solution such as SPLE could conflict with such goals. This aligned with the findings made by Chueca et.al, who state: "*The process of game development is highly iterative, and the design is made while implementing the software itself . This leads to a dearth of non-agile methodologies for video game development"* [4]. These findings further strengthened claims made by focus group participants, which highlighted the importance of creating a flexible integration of SPLE to support the "*chaotic*" and changing nature of game development.

This suggested that for SPLE to gain relevance in game development, an adaptable and flexible solution must be developed to better support speed and innovation.

6 Discussion

This thesis paper aims to investigate the use of Software Product Line Engineering in game development. Through the survey methods, there were many valuable insights gathered from experienced and non-experienced developers. This chapter will present how said insights answer the research questions.

6.1 Research questions addressed

6.1.1 RQ1: What do game developers think about using the Software Product Lines Engineering (SPLE) approach for game engineering?

Most of the results indicate that even though the awareness of SPLE is low, especially amongst less experienced developers, there is clear interest in SPLE alongside positive attitudes towards adopting it in future endeavors. The professional developers who've had direct experience in using SPLE acknowledged the realized benefits (Modularity, cost-effectiveness, and Quality improvements), but they also highlight the rigid structure of SPLE as a barrier. We believe that this shows a deeper mismatch between SPLE's structure-driven nature and the change-driven, creative workflow that defines game engineering [4]. This suggests that for Software Product Line Engineering to be used, it should be adapted into hybrid tools that can be integrated within Agile and Iterative techniques, instead of replacing them.

6.1.2 RQ2: What are the key challenges or obstacles in the use of the SPLE approach in game engineering?

The key challenges identified as mentioned previously in section 4, results are the following:

1. **High initial costs**. Setting up a product line is expensive and requires a significant amount of resources to set up core assets and infrastructure.
2. **Time required to set up**. Creating a Software Product Line requires an upfront time investment to develop the core architecture and reusable assets before any individual game can be finished.
3. **Management complexity**: The variability of game assets (Graphics, User Interface, Physics, etc.) across different products and platforms requires careful management. Without a planned strategy for managing this variability, software maintenance can become very difficult.
4. **Organizational resistance**: Due to the investment required by SPLE, organizations rely on already accustomed software development methods, such as Agile or Iterative techniques. There is a perceived risk changing from immediate and fast methods to a slower but long-term systematic reuse strategy.

Finally, the structure of SPLE can come across as a barrier of creativity, especially in a domain that changes rapidly and favors creativity, such as game development. Several survey participants noted that if a bug is discovered in a reusable component, implementing a bugfix becomes complicated as it must be compatible with all products of that product line.

6.1.3 RQ3: How can these challenges be addressed or mitigated?

Many of the gaming professionals who participated in the questionnaire outlined two methods and ways to address the challenges. These suggestions are:

1. **The development of more dynamic SPLE tools**. This directly addresses the challenges of management complexity andthe structured nature of Software Product Line Engineering. Developing user-friendly tools that seamlessly

integrate with popular game engines such as Unity or Unreal could make variability management less complex.

2. **Integrating SPLE tools within more commonly used development methods**. This strategy is more of a gradual integration of Software Product Line Engineering, rather than a direct implementation. SPLE principles could be introduced as plugins or features within Agile or Iterative workflows. This could help lessen organizational resistance by minimizing upfront investments, allowing them to gradually adopt SPLE.

Moreover, the thematic analysis revealed a consistent suggestion which was that SPLE adoption could increase if it was perceived as a flexible tool and not a rigid system in order to be adaptable to the creative and fast nature of game development. For SPLE to be accepted, it must be framed as a modular tool that developers can use in various sections of a project (Physics engines, UI libraries, etc.), instead of a complete rework of the system's architecture.

6.1.4 RQ4: If given proper training for the use of SPLE, will game developers learn and use the SPLE approach for game development?

The majority of results gathered from the survey methods support this question. All of the participants of the focus group that were unaware of SPLE expressed willingness to learn more and potentially adopt it for future work if given proper training. Furthermore, a large subset of questionnaire respondents followed this theme, expressing enthusiasm to adopt SPLE due to the benefits it provides for developing games.

However, the data suggests that for training to be effective and lead to sustained adoption of SPLE, it must address the specific challenges associated with it. It must reframe Software Product Line Engineering not only as a structured process, but also as a flexible set of principles or tools that can be adapted into more agile environments. Thus, it should focus on how to apply SPLE on various subsystems within the project, such as physics or graphics.

6.2 Comparisons with previous works

Some of the results gathered in this study extend the findings from related works. For example in the case of [1], it showcases how the SPLE approach leads to more correct and efficient results when compared against the Clone and Own approach. Furthermore, [2] showcases how SPLE could improve code reuse and improve development time. The findings of these previous works can be compared to the results from this thesis, where the developers who answered the questionnaire similarly reported that using SPLE reduces development time, provides customization support, and improves reusability. Moreover, as seen in the Model-Driven Game Development (MDGD) approach of the paper [3], the trade off between abstraction and code familiarity identified by the authors is similar to some of the concerns raised by participants of this study. Most of the respondents from the survey methods mention issues that the SPLE approach could bring a level of complexity when understanding shared codebases for teams unfamiliar with the product line's architecture, especially when the variability amongst the products is high. These concerns suggest that whilst SPLE can improve organizational structure, it may limit developer flexibility and creativity [4]. Thus, this thesis supports the results of previous works.

6.3 Generalization & Limitations

This study explores the use and perception of Software Product Line Engineering (SPLE) in Game development. Hence, the results offer insights into the potential benefits, as well as the perceived limitations associated with SPLE use, especially amongst small to medium-sized development teams. However, some limitations affect how these findings can be generalized beyond this specific study.

The study relies upon a single survey method, which contains an online questionnaire and one focus group. That limits this thesis's overall breadth. Whilst triangulation between methodologies and literature was used to validate patterns and solidify claims backed by proven facts, a broader selection of methodologies would strengthen the overall reliability.

To conclude, whilst the findings of this study apply to game development they might not apply to other areas of software development, studies or research papers without the necessary validation. Nevertheless, they are still relevant and helpful for the area of software architectures and game development, particularly in providing knowledge and understanding in how reuse oriented approaches such as SPLE can support modularity and scalability in a family of products.

6.4 Threats to Validity

This thesis presents some potential threats to validity that should be taken into consideration when interpreting the results.

6.4.1 Internal Validity

Internal validity refers to if the results can be trusted, or if they were affected by biases.

- **Selection Bias**: The sample group used to gather data may over- or under-represent certain groups (e.g., indie developers, academics, experienced game developers), which could limit the generalizability of the findings.
    - To mitigate this, participants were recruited from a large variety of channels, including forums, academic networks, and social media to gather diverse perspectives.
- **Experience Variability**: Developers' knowledge about SPLE could differ, thus it could influence their perceptions and responses.
    - To mitigate this, both survey methods started with a short yet concise introduction to SPLE, ensuring that all participants started with a shared foundational understanding before giving their answers.

6.4.2 Construct Validity

Construct Validity aims to evaluate whether the study successfully measured what it set out to measure.

- **Ambiguity in questions**: There could be ambiguity in survey questions that could lead to misinterpretation, which may influence responses given.
    - To mitigate this threat, all questions were analyzed by peers and university professors to refine and ensure clarity in each question posed.
- **Simplified SPLE definition**: The introduction for Software Product Line Engineering for all survey methods might not provide a full and necessary explanation, leading to uncertainty and misunderstandings in responses.
    - To mitigate this, the introduction was carefully written based on previous literature and included examples to help understand the method better.

6.4.3 Reliability

Reliability refers to the replicability of this study's results.

- **Small sample size**: The limited number of participants (16 for the questionnaire, 6 for the focus group) might reduce the generalizability of the findings.

    - To mitigate this, the sample was targeted, and provided their personal insights. Due to the multi-method survey approach taken (focus group & questionnaire), quantitative trends could be validated through the focus group, strengthening the findings despite the small sample size.
- **Time constraints:** The timeframe for conducting the study and gathering results was limited, which in turn reduced the opportunity to reach a larger number of participants or apply alternative data collection techniques
    - To mitigate this, the study focused on the survey methods (questionnaire and focus group) to provide depth within the available time. Moreover more effort was placed into collecting high quality responses over larger amounts.

7 Conclusions and Future Work

This thesis aimed to investigate the use, potential and perception of Software Product Line Engineering in game development. The motivation came from the growing need for modularity and efficiency in game development. To address these concerns, this study focused on four research questions:

RQ1): What do game developers think about using the Software Product Lines Engineering (SPLE) approach for game engineering?

RQ2): What are the key challenges or obstacles in the use of the SPLE approach in game engineering?

RQ3): How can these challenges be addressed or mitigated?

RQ4): If given proper training for the use of SPLE, will game developers learn and use the SPLE approach for game development?

The findings gathered from the questionnaire and focus group show that the awareness and adoption rate of Software product line engineering(SPLE) in game development is quite limited. The developers who have experience working with SPLE provide benefits such as improvement in consistency, reduced development time, and cost efficiency, but is balanced by challenges such as high initial setup costs, tool limitations, and difficulties in managing variability. Moreover, developers with limited or no experience of SPLE showed interest in learning more and possibly adapting it for future projects. Whilst SPLE is suitable in theory, the creative nature of game development might limit the integration of SPLE if no dynamic tools are created. The results gathered in this thesis contribute practical value by clarifying the disconnect between the theoretical benefits and the practical application. Nevertheless, these findings are collected from a focused sample and need to be interpreted with caution when applied to larger domain or non software fields.

7.1 Limitations and reflection

If given more time to research, this study could have expanded in the methodological scope and participant number. For example, there could have been a broader range of data gathering methods such as; interviews with experienced developers working on bigger game studios that could have improved the findings with deeper perspectives on SPLE adoption.

Furthermore, if given more time for data collection, the questionnaire and focus groups could have been refined to include more interactive elements and reach a larger respondent amount. Nevertheless, whilst this study provides insights, the refinements and additional time could have improved the analysis and extended the applicability of the results.

7.2 Future work

This study mainly focuses on the use and perception of SPLE in game development. Thus, the findings can be used to inspire future research. The work that could be done as an extension of the results found on this thesis include:

- Developing educational media or competitions for teaching SPLE to developers to spread awareness.
- Case studies of gradual SPLE adoption in game studios to explore the organizational change.
- Developing a dynamic and flexible tool that utilizes SPLE principles for code reuse and modularity. This could then be tested to evaluate such a solution.

References

[1] J. Chueca, Jose Ignacio Trasobares, Á. Domingo, L. Arcega, C. Cetina, and J. Font, "Comparing software product lines and Clone and Own for game software engineering under two paradigms: Model-driven development and code-driven development," *Journal of systems and software/The Journal of systems and software*, vol. 205, pp. 111824–111824, Nov. 2023, doi: https://doi.org/10.1016/j.jss.2023.111824.

[2] Emad Albassam and H. Gomaa, "Applying software product lines to multiplatform video games," May 2013, doi: https://doi.org/10.1109/gas.2013.6632582

[3] B. Neto, L. Fernandes, C. Werner, and Jano, "Reuse in Digital Game Development," Dec. 2009, doi: https://doi.org/10.1109/icut.2009.5405710

[4] J. Chueca, J. Verón, J. Font, F. Pérez, and C. Cetina, "The consolidation of game software engineering: A systematic literature review of software engineering for industry-scale computer games," *Information and Software Technology*, vol. 165, p. 107330, Jan. 2024, doi: https://doi.org/10.1016/j.infsof.2023.107330.

[5] C. W. Krueger, "Software reuse," *ACM Computing Surveys (CSUR)*, vol. 24, no. 2, pp. 131–183, Jun. 1992, doi: https://doi.org/10.1145/130844.130856.

[6] X. Chen, M. Usman, and Deepika Badampudi, "Understanding and evaluating software reuse costs and benefits from industrial cases—A systematic literature review," *Information and Software Technology*, vol. 171, pp. 107451–107451, Jul. 2024, doi: https://doi.org/10.1016/j.infsof.2024.107451.

[7] K. Pohl, Günter Böckle, and F. Linden, "Software Product Line Engineering," *SpringerLink*, 2023, doi: https://link.springer.com/book/10.1007/3-540-28901-1.

[8] T. Kehrer, T. Thum, A. SchultheiB, and Paul Maximilian Bittner, "Bridging the Gap Between Clone-and-Own and Software Product Lines," *OPen Access Repositorium der Universität Ulm (OPARU) (Ulm University)*, May 2021, doi: https://doi.org/10.1109/icse-nier52604.2021.00013.

[9] H. Engström, B. Berg Marklund, P. Backlund, and M. Toftedahl, "Game development from a software and creative product perspective: A quantitative literature review approach," *Entertainment Computing*, vol. 27, pp. 10–22, Aug. 2018, doi: https://doi.org/10.1016/j.entcom.2018.02.008.

[10] "Living With Technical Debt—A Perspective From the Video Game Industry | IEEE Journals & Magazine | IEEE Xplore," *ieeexplore.ieee.org*. https://ieeexplore.ieee.org/document/9508260

A Appendix 1

Questionnaire

You can view all the questions within the questionnaire here:

Demographics

Q1.1) What is your role in your organization?
Q1.2) For how long have you been working as a game developer?
Q1.3) How many games have you participated in developing?
Q1.4) How many people are working in your company/team?
Q1.5) What genre of games do you/your team or company create?
Q1.6) What game engine do you/your team or company use for game development?

SPL usage filter

Q1.7) Before answering this questionnaire, were you familiar with the Software Product Line Engineering (SPLE) approach?
Q1.8) Please briefly describe your understanding of SPLE.
Q1.9) Have you used SPLE for game development?

Developers that have used SPLE for game development

Q2.1) What has been your, your team, or your company's motivation for using SPLE for games development?
Q2.2) What are the benefits you, your team or company has got from using SPLE in game development?
Q2.3) What are the main challenges or drawbacks you realized with the use of SPLE for game development?
Q2.4b) What problems or issues did you, your team or company encounter when using SPLE approach for game development?
Q2.5) Do you think or perceive the use of hybrid approaches that combine use of core SPLE principles with other development or engineering methods for game development?
Q2.5b) Please describe in what way you think that the use of core SPLE principles can be combined with other game development or engineering methods.

Developers that have not used SPLE for game development

Q3.1) For what reasons do you think you, your team, or company are not using SPLE for game development?
Q3.2) What actions or steps do you think may help in increasing the use of SPLE approach in the game development?
Q3.3) Which engineering processes or development techniques are you, your team or company currently using or have used for game development?
Q3.4) Do you see any limitations, problems, or issues with your current engineering or development methodologies for game development?
Q3.4b) What limitations, problems, or issues have you or your team have experienced

with your current engineering or development methodologies for game development?
Q3.5) Do you see or think of any opportunities or benefits of using SPLE for game development in your work?
Q3.5b) What opportunities or benefits do you see or think of for use of SPLE for game development in your work?
Q3.5c) Why do not you see or think of any opportunities or benefits of using SPLE for game development in your work?
Q3.6) In future, would you like to use SPLE approach for game development?
Q3.6b) Why would you like to use SPLE approach for game development in future?
Q3.6c) Why would you not like to use SPLE approach for game development in future?

Concluding thoughts

Q4.1) Do you think with more awareness and exposure, the use of SPLE in game development can be increased?
Q4.1b) Why do you think that the use of SPLE in game development can not be increased even with more awareness and exposure?
Q4.2) What other suggestions or recommendations do you have for improving the use of SPLE in the game industry?
Q4.3) Do you think the use of SPLE in game engineering will increase in the future?
Q4.4) For the above questions, explain why you think that the use of SPLE in game engineering is going to be **increased or not to be increased.**
Q4.5) How confident are you in general about the answers you gave in this questionnaire? (higher stars indicate higher confidence and vice versa)